CXL RAS Emulation: Mistakes, Blockers & Solutions

Overview: This document records every significant blocker encountered while setting up a CXL RAS emulation environment on WSL2 + QEMU. The setup session spanned many hours largely due to these issues. This document details each mistake and its root cause to ensure a thorough understanding of the CXL RAS, serving as a troubleshooting reference.

Mistake Categories Covered

- WSL2 kernel limitations
- QEMU version and implementation gaps
- Guest kernel configuration gaps
- CXL event delivery mechanisms
- Environment and terminal environment segregation

Mistake 1: Assuming WSL2 Could Perform EINJ Natively

The project requires Reliability, Availability, and Serviceability (RAS) error injection. The initial approach utilized APEI EINJ (ACPI Error Injection), the standard Linux kernel interface for injecting hardware errors, under the assumption that WSL2 would natively support it.

Error / Manifestation:

```
ls /sys/kernel/debug/apei/einj
ls: cannot access '/sys/kernel/debug/apei/einj': No such file or directory
```

Root Cause:

WSL2 runs on top of the Hyper-V hypervisor, which does not expose ACPI EINJ tables to the WSL2 virtual machine. EINJ relies on system firmware (ACPI BIOS) to supply an EINJ table that explicitly describes how to inject errors into the underlying hardware. Because Hyper-V omits this table, even a properly compiled kernel has no underlying interface to target.

Solution / Fix:

Utilize QEMU-side error injection via the QEMU Machine Protocol (QMP) instead of system-level EINJ. Since QEMU emulates the CXL device entirely, errors can be injected directly into the emulated hardware layer via QMP. This approach works better for our project for an emulation-focused project, as the primary objective is validating the CXL driver stack rather than testing the host firmware injection infrastructure.

Key Takeaway:

EINJ is built specifically for real physical hardware. For virtualized emulation environments, QMP injection is the correct, intended approach.

Mistake 2: Utilizing Stock System QEMU (Version 8.2.2) Instead of Building Version 9.0

The default Ubuntu package manager installs QEMU version 8.2.2. This package was initially deployed without verifying subsystem compatibility.

Error / Manifestation:

```
cxl region: create_region: decoder3.0: set_mode failed: No such device or
address
cxl region: cmd_create_region: created 0 regions
```

Root Cause:

The CXL emulation layer in QEMU 8.2.2 is incomplete. The 'set_mode' command issued by the Linux CXL driver to configure the endpoint decoders fails because the emulated device does not respond correctly to the 'Set Partition Info' mailbox command. QEMU 9.0.0 introduced critical, updated architectures for proper CXL hardware emulation.

Solution / Fix:

Compile QEMU 9.0.0 directly from source and update the environment launch script to target the newly built binary:

```
sed -i 's|^qemu-system-x86_64|/home/user/qemu/build/qemu-system-x86_64|'
~/cxl/start-cxl.sh
```

Key Takeaway:

We learned that we must always verify the software version baselines before trying to implement or test new hardware specs. CXL emulation features are undergoing rapid development and see significant enhancements between minor emulator updates.

Mistake 3: Stock QEMU 9.0 Lacks Set Partition Info Mailbox Command Implementation

Even after upgrading to source-built QEMU 9.0.0, the CXL memory region creation process continued to fail with identical errors.

Error / Manifestation:

```
cxl region: create_region: region0: failed to commit decode: No such device
or address
libcxl: write_attr: failed to write 1 to
/sys/bus/cxl/devices/root0/decoder0.0/region0/commit: No such device or
address
```

Root Cause:

While QEMU 9.0.0 brought massive improvements, it still does not natively implement the 'Set Partition Info' CXL mailbox command (opcode 0x4101). The Linux kernel's CXL subsystem issues this command to configure how device memory splits between volatile and persistent modes. Without this handler, the driver cannot successfully commit and bring CXL memory regions online. This was confirmed by auditing the emulator source code:

```
grep -r "set_partition\|SET_PARTITION" ~/qemu/hw/mem/cxl* | head -20
# [No output returned - command is unimplemented]
```

Solution / Fix:

Manually patch the QEMU source code to handle the missing mailbox command. The patch introduces:

- A new definition for the command opcode: #define SET_PARTITION_INFO 0x1
- An implementation function named cmd_ccls_set_partition_info that updates the device's interior vmem_size and pmem_size structures.
- Registration of the command within the internal CCLS command dispatch table: [CCLS][SET_PARTITION_INFO] = { "CCLS_SET_PARTITION_INFO", cmd_ccls_set_partition_info, 0x11, IMMEDIATE_CONFIG_CHANGE },

The complete patch layout is fully documented in the accompanying Setup Guide.

Key Takeaway:
This showed us that open-source emulators often lack features that the active Linux kernel drivers already expect to be present. When the guest kernel cannot communicate with our emulated hardware, our best approach to reaching our goals is to extend the emulator code directly rather than trying to modify the kernel.

Mistake 4: Attempting to Manually Configure CXL Regions via raw sysfs Commands

Prior to exploring specialized userspace configuration utilities, several attempts were made to initialize CXL memory regions by interacting directly with raw sysfs attributes, resulting in highly ambiguous kernel faults.

Error / Manifestation:

```
bash: echo: write error: No such device or address
bash: echo: write error: Invalid argument
bash: echo: write error: Device or resource busy
tee: /sys/bus/cxl/devices/region0/uuid: Permission denied
```

Root Cause:

The kernel sysfs control layout for CXL region topology is strictly sequential and exceptionally sensitive. A valid configuration transaction demands a rigid, precise operational sequence: (1) instantiate the region name, (2) verify or write the auto-populated UUID, (3) establish interleave granularity, (4) declare interleave ways, (5) allocate the size parameter, (6) assign the target endpoint mappings, and finally (7) commit the execution status. Any out-of-order interaction causes generic, unhelpful VFS write errors.

Solution / Fix:

Deploy the standard ndctl orchestration package and utilize the specialized cxl tool wrapper instead of raw sysfs manipulation:

```
sudo apt install -y ndctl
cxl create-region -d decoder0.0 -m mem1 -s 512M -t ram
```

This single abstracted command executes the entire 7+ step lifecycle automatically, checking state constraints and gracefully handling edge-case errors at each stage.

Key Takeaway:

When interacting with multi-layered kernel subsystems, always prioritize the official userspace management utilities over manual sysfs parameter tuning. The cxl package serves as the standard development toolchain used by kernel engineers.

Mistake 5: Deploying a Stock Ubuntu Cloud Image Distribution Kernel

The standard Ubuntu cloud OS image (noble-server-cloudimg-amd64.img) defaults to a generic distribution kernel (e.g., 6.8.0-generic). Running the cxl utility against this baseline system triggered core dependency dropouts.

Error / Manifestation:

```
modprobe: FATAL: Module cxl_core not found in directory /lib/modules/6.8.0-110-generic
cxl region: create_region: decoder3.0: set_mode failed: No such device or address
```

Root Cause:

Stock distribution kernels explicitly strip out or omit experimental and specific hardware validation symbols. The vendor kernel lacks several vital compilation flags necessary for full CXL exploration:
• CONFIG_CXL_REGION_INVALIDATION_TEST: Omission causes cache synchronization routines to fail.
• CONFIG_MEMORY_FAILURE: Essential foundational flag; without it, downstream MCE handlers cannot resolve.
• CONFIG_CXL_MCE: Disables delivery of device error signals to the main kernel Machine Check Exception architecture.
• CONFIG_CXL_FEATURES: Unset by default.
Furthermore, background system updates (unattended upgrades) automatically overwrite the running kernel directory structures, destabilizing manually built modules.

Solution / Fix:

Compile a dedicated CXL research kernel sourced from the official upstream CXL maintainers' tree (Linux 7.0.0 layout). Ensure all necessary debugging features are explicitly enabled. Configure QEMU to boot this custom bzImage directly via the '-kernel' command-line directive, circumventing the guest distribution's local bootloader completely.

Key Takeaway:

Standard client or server distribution kernels are not compiled to support deep architectural CXL evaluations. For low-level RAS prototyping, we must maintain a dedicated kernel built directly from the subsystem maintainers' source repositories.

Mistake 6: Launching QEMU with TCG Software Acceleration Instead of KVM

The initial virtualization environment script launched QEMU using the software emulation flag: -accel tcg,thread=multi. This introduced two severe structural bugs.

Performance / Operational Impact:

- 1. Extreme Latency: TCG software translation caps guest code execution at roughly 5–10% of physical system speed. A routine 113MB package installation expanded into a 40-minute process.
- 2. Broken Interrupt Trees: CXL hardware error tracking counts on MSI-X vector signaling. Under software TCG execution, these hardware interrupt boundaries are not properly emulated, preventing QMP-injected errors from ever alerting the guest kernel driver.

Error / Manifestation:

```
# Under TCG Emulation (Interrupt counters remain static during injection):
cat /proc/interrupts | grep "0e:00"
45: 0  0  0  0  PCI-MSIX-0000:0e:00.0

# Under KVM Acceleration (Counters increment properly upon error
injection):
cat /proc/interrupts | grep "0e:00"
40: 0  0  0  1  PCI-MSIX-0000:0e:00.0
```

Root Cause:

The Tiny Code Generator (TCG) implementation lacks the high-fidelity pipeline emulation required to accurately fire the MSI-X vector pathways utilized by advanced CXL endpoints. KVM utilizes native hardware virtualization extensions (Intel VT-x / AMD-V), which natively and correctly marshal PCI MSI-X hardware assertions straight into the guest interrupt controller. Without functioning interrupts, the guest driver remains completely unaware that new records have arrived in the device's internal event logs.

Solution / Fix:

Expose and activate native KVM acceleration capabilities inside the host WSL2 environment, and update the execution flags:

```
# 1. Install and load KVM modules within WSL2 host:
cd ~/WSL2-Linux-Kernel
sudo make modules_install
sudo modprobe kvm && sudo modprobe kvm_intel

# 2. Modify QEMU launch scripts:
Change from '-accel tcg,thread=multi' to '-accel kvm'
```

Key Takeaway:

KVM acceleration is a strict architectural requirement for correct low-level hardware emulation, not merely a speed optimization. For features involving real-time hardware interrupt tracking, software emulation layers like TCG are insufficient.

Mistake 7: CONFIG_CXL_MCE Silently Omitted from Kernel Compilation

Even after resolving KVM dependencies and proving that MSI-X interrupts were physically firing, injected CXL errors still failed to output to standard kernel diagnostic channels.

Error / Manifestation:

```
cxl_pci 0000:0e:00.0: CXL MCE unsupported
```

Root Cause:

The kernel parameter CONFIG_CXL_MCE is declared as a conditional 'def_bool' inside the Kconfig subsystem. It only auto-activates if both CONFIG_X86_MCE and CONFIG_MEMORY_FAILURE are fully enabled. While the main x86 MCE flag was set, CONFIG_MEMORY_FAILURE was absent. Consequently, the configuration toolchain silently dropped the CXL MCE handler entirely and compiled a non-operational stub function instead:

```
static inline int devm_cxl_register_mce_notifier(...) {
    return -EOPNOTSUPP; // This fallback stub was executing silently
}
```

Solution / Fix:

Explicitly toggle the missing structural dependency within the guest kernel's baseline configuration and run a complete rebuild cycle:

```
scripts/config --enable CONFIG_MEMORY_FAILURE
make olddefconfig
make -j$(nproc)
cp arch/x86/boot/bzImage /mnt/c/Users/User/cxl_guest_kernel
```

Key Takeaway:

When troubleshooting quiet dropouts in conditional kernel subsystems, map the complete dependency tree manually inside the Kconfig files. The build chain will not throw an explicit warning when a feature is omitted; it will simply fallback to dummy stub code.

Mistake 8: Inspecting dmesg for Hardware Events Instead of tracing channels

After establishing a pristine build configuration and ensuring QEMU successfully ingested error payloads, standard system logs (dmesg) still returned no diagnostic output concerning the injected runtime events.

Root Cause:

By design, the Linux kernel CXL subsystem does not print transactional hardware errors directly to the primary kernel ring buffer (dmesg). Instead, it routes them directly into the high-performance kernel tracing subsystem (ftrace). The error details are handled as structured trace packets rather than plain-text log messages.

Solution / Fix:

Activate the specific ftrace infrastructure channels for the CXL driver and monitor the dynamic runtime trace stream:

```
echo 1 > /sys/kernel/debug/tracing/events/cxl/enable
echo 1 > /sys/kernel/debug/tracing/tracing_on
cat /sys/kernel/debug/tracing/trace_pipe &
```

Injected events will immediately stream to the active console terminal session:

```
irq/40-0000:0e:-77 [003] 324.455313: cxl_general_media: memdev=mem0
host=0000:0e:00.0 serial=2 log=Informational
```

Key Takeaway:

Different kernel layers utilize vastly different logging facilities. High-frequency telemetry and hardware events are sent to ftrace rather than system logs. Check under '/sys/kernel/debug/tracing/events/' to explore all available kernel subsystem tracing targets.

Mistake 9: Failing to Pin the Guest Kernel Package Version

During a normal infrastructure reboot sequence, the underlying Ubuntu package manager pulled a background security patch, automatically bumping the distribution kernel from version 6.8.0-110 to 6.8.0-111.

Error / Manifestation:

```
modprobe: FATAL: Module cxl_core not found in directory /lib/modules/6.8.0-
111-generic
```

This silent background configuration shift invalidated the pre-built 'linux-modules-extra' packages, necessitating a lengthy re-download cycle inside the highly unoptimized guest environment.

Solution / Fix:

Explicitly set an apt-level package lock immediately following the initial development environment setup to prevent automated package upgrades:

```
sudo apt-mark hold linux-image-$(uname -r) linux-modules-$(uname -r) linux-modules-extra-$(uname -r)

# Verify status:
sudo apt-mark showhold
```

Key Takeaway:

Within experimental system engineering environments utilizing customized modules, always pin our precise kernel baseline versions. Unmanaged repository updates can break kernel configurations and invalidate hours of low-level optimization work.

Mistake 10: Assuming Poison Memory Addresses Match Local Device Boundaries (DPA vs SPA)

Early injection attempts aimed directly at base offset 0 ('start': 0) within the device. However, no memory poison events were caught by the kernel, despite the memory region being active and bound.

Root Cause:

The CXL memory target maps into a high system memory boundary (e.g., System Physical Address or SPA: 0x110000000 = 4563402752 decimal). The Linux kernel tracking frameworks only monitor and trace memory faults that fall within properly mapped, actively committed system boundaries. Injecting an error at address 0 targets memory outside of the operating system's active CXL map, causing the kernel to ignore the event.

Solution / Fix:

Always audit the live operational system map to extract the exact base SPA address prior to injecting error parameters via QMP:

```
# Extract the true system resource address mapping inside the guest:\cat /sys/bus/cxl/devices/region0/resource
# Returns: 0x110000000
```

```
# Convert hexadecimal to base-10 decimal for QMP inputs: 0x110000000 =
4563402752

# Issue injection with explicit SPA parameters via the host QMP terminal:
{"execute": "cxl-inject-poison", "arguments": {"path":
"/machine/peripheral/cxl1", "start": 4563402752, "length": 64}}
```

Key Takeaway:

Device Physical Addresses (DPA) in QEMU start at base 0, but System Physical Addresses (SPA) represent the true coordinates used by the kernel memory management subsystem. Always align error injections with the active SPA range.

```
echo '{"execute": "query-commands"}' | nc 127.0.0.1 4444 | python3 -m
json.tool | grep cxl
```

Key Takeaway:

Never assume feature parity across emulator revisions. Always query the engine's active runtime capabilities directly before structuring test scripts around specific QMP commands.

Summary: System Triage Decision Tree

When injected CXL hardware errors are not appearing in the guest tracking tools, run through the following sequence:

- 1. Verify Virtualization Acceleration: Ensure QEMU is utilizing KVM. (TCG fails to process MSI-X signals). Fix: Apply '-accel kvm'.
- 2. Audit Subsystem Modules: Verify CXL drivers are active via 'lsmod | grep cxl'. Fix: Run 'modprobe cxl_core cxl_pci'.
- 3. Check Memory Regions: Ensure a CXL mapping is committed using 'cxl list -R'. Fix: Execute 'cxl create-region'.
- 4. Diagnose Kernel Stubs: Check for 'CXL MCE unsupported' faults. Fix: Enable CONFIG_MEMORY_FAILURE and compile.
- 5. Inspect Vector Interrupts: Monitor '/proc/interrupts' to ensure counts increment during injection. Fix: Verify KVM module status on host.
- 6. Redirect Diagnostic Logging: Ensure we are monitoring ftrace pipes rather than standard dmesg outputs. Fix: Read from '/sys/kernel/debug/tracing/trace_pipe'.
- 7. Validate Target Coordinates: Ensure injection addresses match the system memory map (SPA). Fix: Query region resource coordinates and convert to base-10 decimal.